Aonde você quer chegar?

Vai com a

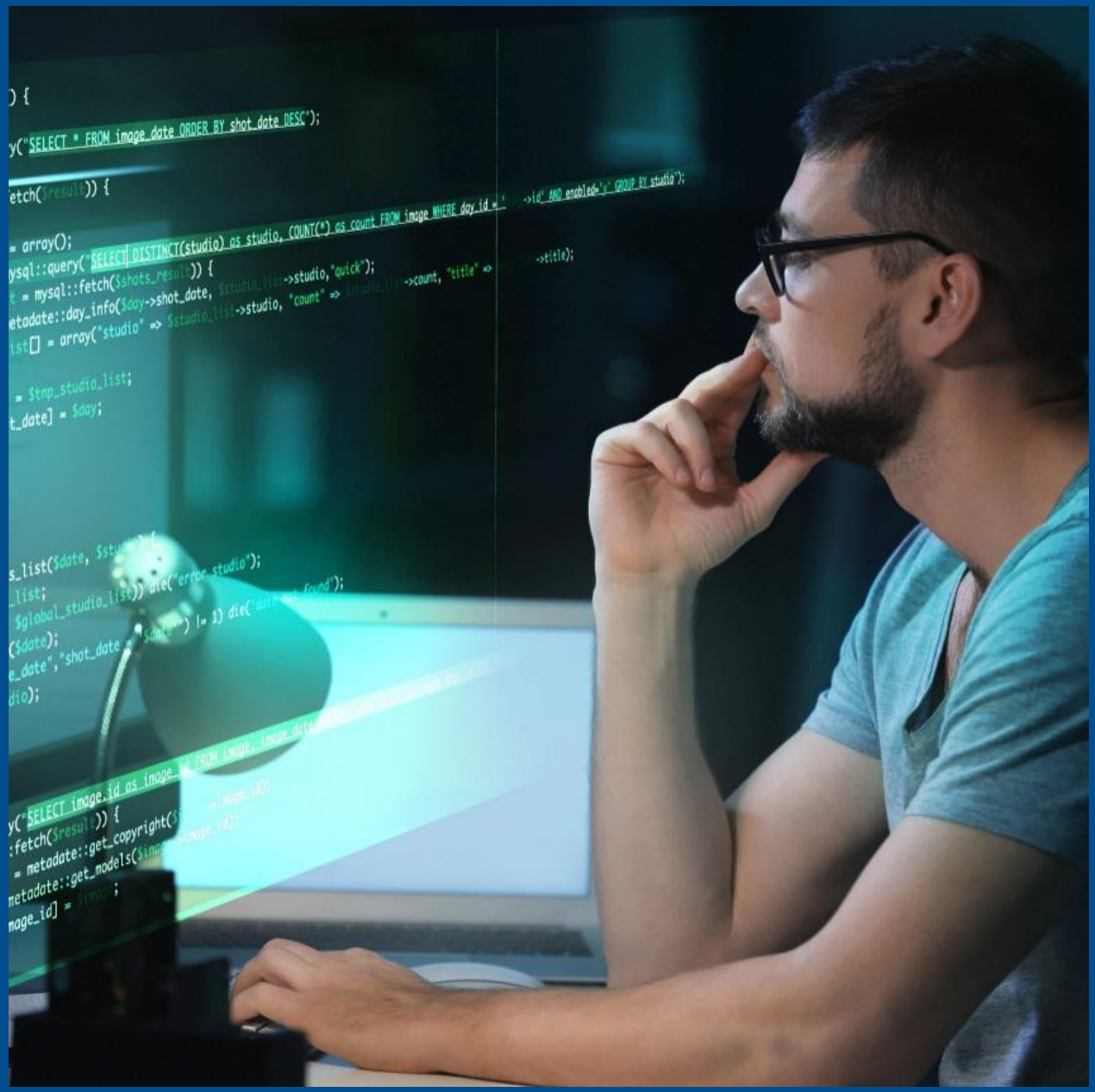
UniCesumar

EAD, SEMIPRESENCIAL E PRESENCIAL

UniCesumar
EAD, SEMIPRESENCIAL E PRESENCIAL

Tecnologias Emergentes em Engenharia de *Software*

Aula 1



Prof. Rômulo Santos

PhD Computação. MSc Engenharia Elétrica. MSc Ciência da Informação.

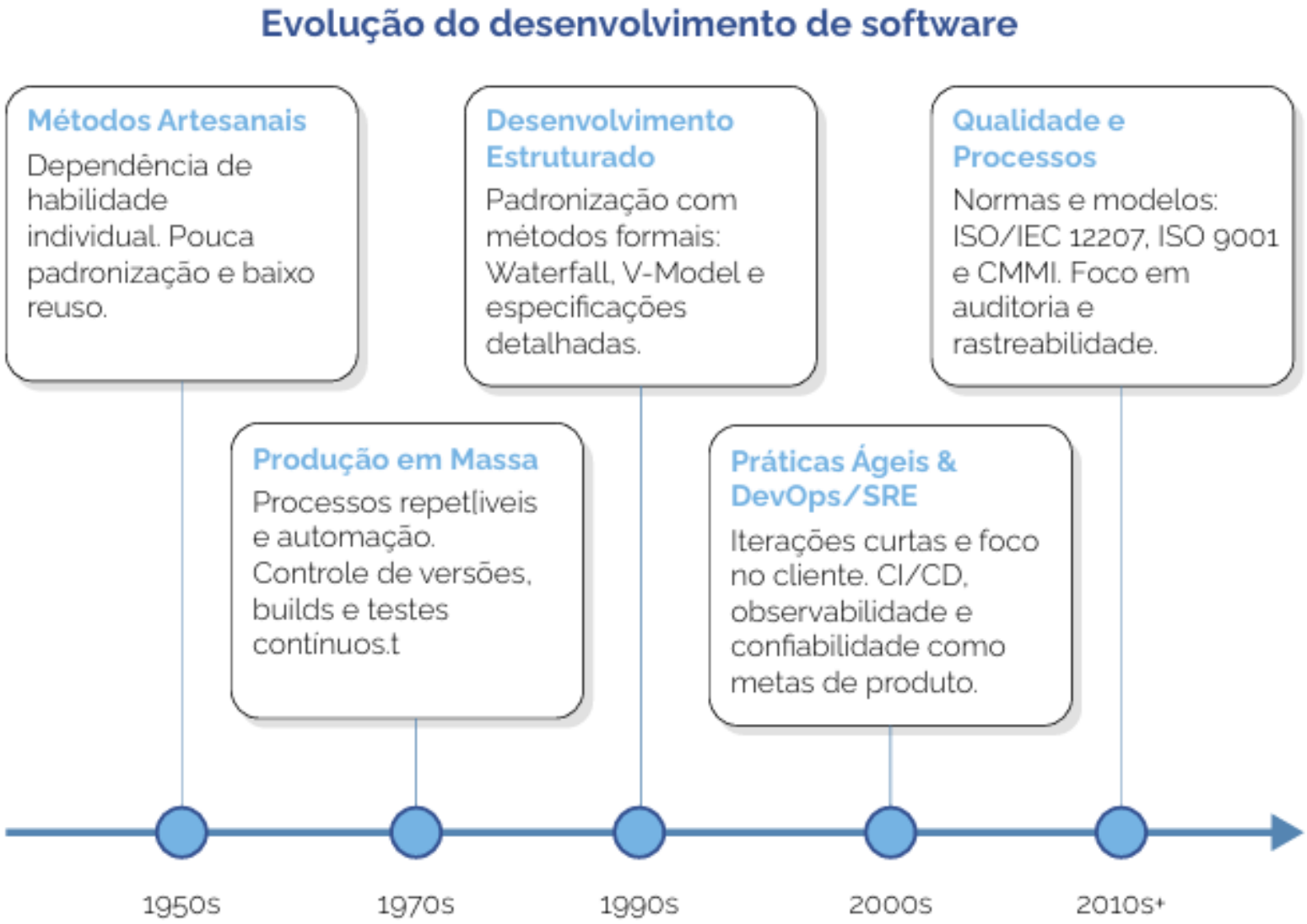
Sumário

- Panorama histórico e consolidação do paradigma.
- Do artesanal à fábrica: por que padronizar?
- Processos repetíveis: como a fábrica opera.
- Fábrica ≠ Metodologia.
- Alinhando o vocabulário: ALM e rastreabilidade.
- Modelos de fábrica.
- Combinar modelos com governança leve.

Panorama histórico e consolidação do paradigma:

A engenharia de *software* percorreu um arco que vai dos métodos artesanais (anos 1950-1960) ao desenvolvimento estruturado (1970-1980), passando por normas e modelos de processo nos anos 1990 (ISO/IEC 12207, ISO 9001, CMMI).

A partir dos anos 2000, a automação e os processos repetíveis prepararam o terreno para abordagens ágeis e, na década de 2010, para DevOps/SRE, com CI/ CD, observabilidade e confiabilidade como metas de produto.



Do artesanal à fábrica: por que padronizar?

No modelo artesanal, as decisões tornam-se tácitas, a qualidade depende de indivíduos e a escalabilidade é limitada. A fábrica de *software* propõe uma plataforma operacional: catálogo de serviços, critérios de entrada/saída, esteiras automatizadas e métricas de fluxo.

Tal plataforma reduz a variação, aumenta a previsibilidade, facilita a melhoria contínua. Em ambientes brasileiros, governamentais e bancários, a adoção trouxe transparência (evidências em *gates*) e rastreabilidade para auditorias; em empresas digitais, viabilizou escala com segurança.

Modelos de produção de softwares: Artesanal x Fábrica

Modelo Artesanal	Modelo de Fábrica
⊗ Processos pouco definidos e documentação irregular	✓ Processos padronizados e bem documentados
⊗ Trabalho individualizado, dependente de habilidades	✓ Trabalho em equipe, doco na repetibilidade
⊗ Dificuldade de escalar e manter a qualidade	✓ Escalabilidade com controle rigoroso de qualidade
⊗ Controle de produtividade informal	✓ Monitoramento contínuo da produtividade

Estudo de Caso: Central de Serviços Digitais — do Artesanal à Fábrica de Software

Aplicando padronização, catálogo de serviços, quality gates, automação e métricas de fluxo

1. Cenário

Uma autarquia pública recebe demandas de manutenção e evolução de sistemas, integrações, correções urgentes e novos relatórios. Cada equipe atendia pedidos de forma diferente, com pouca padronização e forte dependência de pessoas-chave.



2. Situação inicial: modelo artesanal

- Pedidos chegam por e-mail, chat e planilhas soltas
- Processos pouco definidos e documentação irregular
- Trabalho individualizado e dependente de especialistas
- Dificuldade de escalar e manter a qualidade
- Controle de produtividade informal

3. Transformação para fábrica

Catálogo de serviços → Triagem → Desenvolvimento → Testes e quality gates → Entrega → Monitoramento

- Tipos de serviço padronizados
- Critérios de entrada (DoR) e conclusão (DoD)
- Esteira com automação e evidências
- Métricas de fluxo e SLA

4. Artesanal x Fábrica

Comparativo	Modelo Artesanal	Modelo de Fábrica
Processos	Pouco definidos	Padronizados e documentados
Trabalho	Individualizado	Em equipe e repetível
Qualidade	Oscila conforme a pessoa	Controlada por gates e testes
Escala	Limitada	Escalável com previsibilidade
Medição	Informal	Monitoramento contínuo

5. Ações implantadas

1. Criar catálogo de serviços (API, evolução, correção P1, relatório)
2. Definir entradas mínimas e Definition of Done
3. Implantar pipeline com build, testes e publicação
4. Padronizar revisões, documentação e versionamento
5. Acompanhar lead time, % no SLA e reaberturas

6. Resultados observados

- Mais previsibilidade no atendimento
- Redução de retrabalho e reabertura
- Melhor rastreabilidade para auditoria
- Facilidade para escalar a operação
- Maior transparência sobre prazos e qualidade

Lead time ↓ | % SLA ↑ | Reaberturas ↓

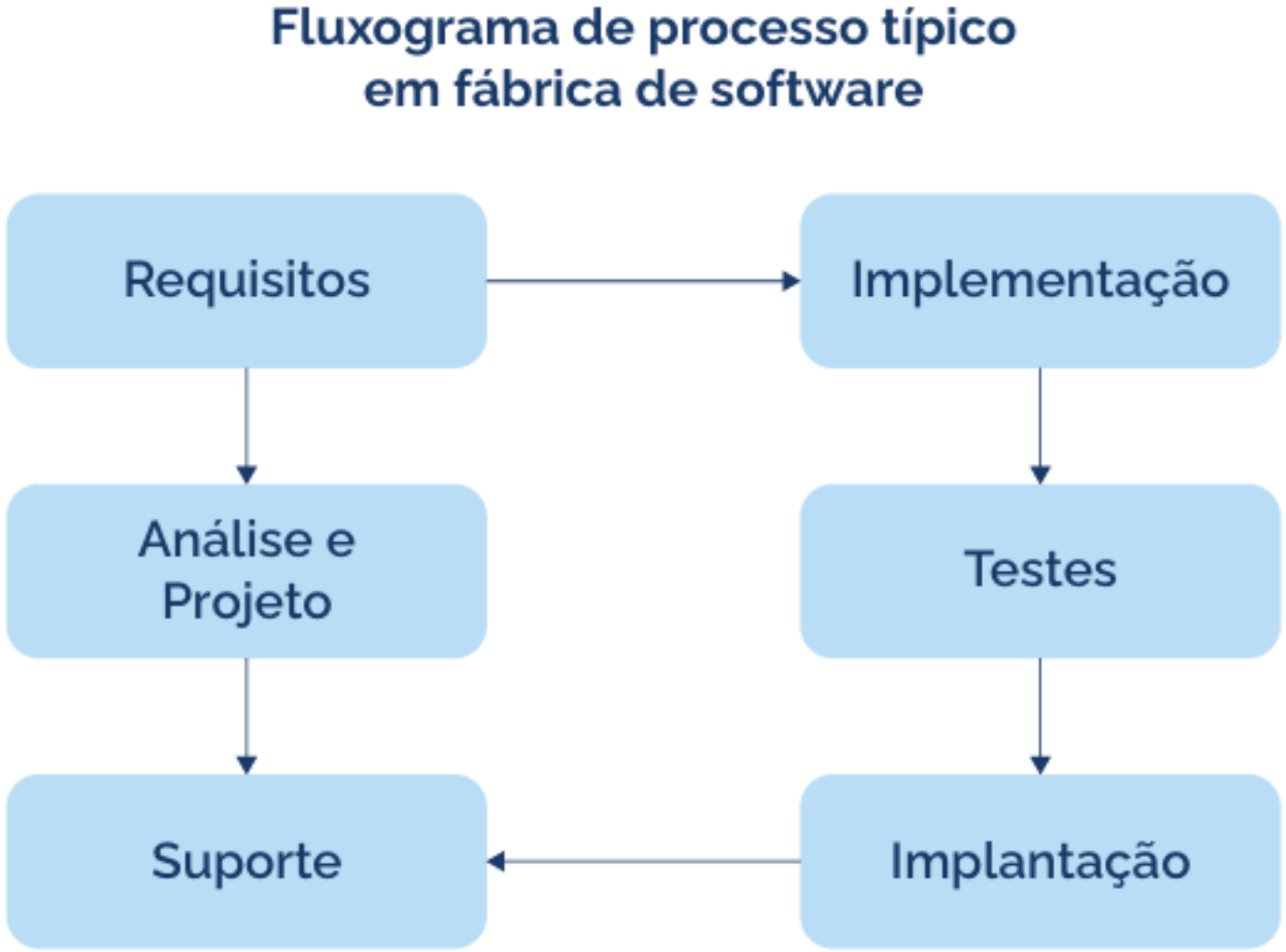
Ideia-chave

Padronizar não significa engessar. Na fábrica de software, a padronização cria linguagem comum, reduz variações e permite escala com qualidade, evidências e melhoria contínua.

Processos repetíveis: como a fábrica opera

Uma fábrica organiza o trabalho em esteiras com *quality gates* que realmente reduzem risco.

Exemplo de fluxo: Requisitos (entrada mínima e DoR) → Análise e Projeto (*impact analysis*, contratos de API/eventos) → Implementação (pequenos PR, *lint*, *build* e testes unitários) → Testes (contratos/integração; poucos E2E para jornadas críticas) → Implantação (CI/CD com *feature flags*, *rollbacks* e verificação *pós-deploy*) → Suporte/Operação (SLI/SLO, *post-mortem* sem culpa).



Processos repetíveis: como a fábrica opera

A **Fábrica de *Software*** organiza o trabalho por meio de uma linguagem comum entre quem solicita e quem desenvolve. O ponto de partida é o **catálogo de serviços**, que transforma demandas genéricas em tipos claros de trabalho, como nova API, evolução de tela, correção crítica, integração externa ou migração de versão.

Cada serviço deve possuir:

- **DoR:** critérios mínimos para iniciar o trabalho;
- **DoD:** evidências necessárias para considerar a entrega concluída;
- **SLA de ciclo:** expectativa de prazo para cada tipo de serviço;
- **Pontos de inspeção:** verificações objetivas durante o fluxo;
- **Limites de WIP:** controle da quantidade de trabalho simultâneo.

A qualidade deve estar incorporada à própria esteira, com **testes automatizados, análise estática, *security scanning*, testes de contrato e validação por *Pull Request***.

Processos repetíveis: como a fábrica opera

Uma *Fábrica de Software* madura utiliza **dados e automação** para acompanhar desempenho, qualidade e confiabilidade do processo.

Principais práticas:

- ***Lead Time*:** mede quanto tempo uma demanda leva da solicitação até a produção;
- **SLA:** verifica se os prazos acordados estão sendo cumpridos;
- **Taxa de reabertura:** indica problemas de qualidade e retrabalho;
- **CI/CD e testes automatizados:** reduzem erros e aceleram entregas;
- **SLI/SLO:** acompanham indicadores como latência, disponibilidade e taxa de erro;
- **Orçamento de erro:** ajuda a equilibrar velocidade de inovação e estabilidade;
- **Reuso e modularidade:** componentes versionados evitam “copiar e colar” soluções;
- **Segurança e conformidade:** LGPD, interoperabilidade e controles automáticos passam a integrar o fluxo.

Estudo de Caso

Fábrica de Software para Serviços Digitais de uma Prefeitura

Como processos repetíveis, automação e qualidade garantem entregas previsíveis e confiáveis

Contexto

Uma prefeitura deseja modernizar seus serviços digitais (ex.: emissão de alvará, IPTU e protocolo eletrônico) para melhorar o atendimento ao cidadão e reduzir custos operacionais.

Demanda

Criar e evoluir um portal de serviços digitais, respondendo rapidamente a novas demandas de secretarias, com qualidade e segurança.

Catálogo de serviços da Fábrica

A fábrica transforma demandas em tipos claros de trabalho:

- Nova API / serviço
- Evolução de funcionalidade
- Correção crítica
- Integração externa
- Migração de versão

Fluxo de processo típico na Fábrica de Software

1 Requisitos

- Entrada mínima (DoR)
- Solicitação registrada no catálogo
- Priorização e SLA

Ex.: Solicitação de nova API para emissão de alvará

2 Análise e Projeto

- Análise de impacto
- Desenho da solução
- Definição de testes e critérios de aceite (DoD)

Ex.: Diagrama, contrato de API, casos de teste

3 Implementação

- Desenvolvimento (pequenos PR)
- Code review
- Linters e análise estática

Ex.: Branch, Pull Request, análise automática

4 Testes

- Testes unitários e de integração
- Testes de contrato
- Security scanning
- Validação de qualidade

Ex.: Execução automática no pipeline (CI) e relatório de testes

5 Implantação

- Pipeline de CI/CD
- Feature flags
- Deploy controlado (blue/green ou canary)
- Rollback

Ex.: Deploy em ambiente de homologação e produção

6 Suporte

- Monitoramento
- Atendimento (SLA)
- Indicadores de operação (SLI/SLO)
- Pós-mortem sem culpa

Ex.: Dashboards de disponibilidade, latência e taxa de erro

Processos repetíveis, automação e *quality gates* em todas as etapas

Resultados obtidos

Após 6 meses de operação da Fábrica de Software, a prefeitura alcançou ganhos significativos:

- **-45%** Tempo de entrega (*lead time*)
- **92%** Taxa de sucesso nos deploys
- **-60%** Incidentes em produção
- **85%** Satisfação dos órgãos solicitantes
- **-30%** Custo de manutenção (reuso e modularidade)

Entregas previsíveis, seguras e de qualidade para o cidadão, com processos padronizados e escaláveis.

Fábrica ≠ Metodologia:

É importante diferenciar **metodologia de desenvolvimento** de **Fábrica de *Software***.

Metodologias como **Scrum, Kanban ou Cascata** organizam a forma como a equipe trabalha no dia a dia, definindo papéis, cadências, artefatos e formas de coordenação.

A **Fábrica de *Software***, por outro lado, atua em uma camada mais ampla da operação. Ela define:

- **O que pode ser solicitado:** catálogo de serviços;
- **Como a demanda entra:** critérios de prontidão - DoR;
- **Como o trabalho flui:** esteiras, automações e integrações;
- **O que significa concluir:** *Definition of Done* - DoD;
- **Como a qualidade é verificada:** *quality gates*;
- **Como o desempenho é acompanhado:** SLA e métricas de fluxo.

Assim, é perfeitamente possível utilizar **Scrum dentro de uma Fábrica de *Software***.

Fábrica ≠ Metodologia:

O **catálogo de serviços** transforma solicitações genéricas em tipos de serviço bem definidos, como:

(1) Evolução de tela; (2) Nova API; (3) Correção crítica; (4) Integração externa; (5) Migração de versão.

Cada serviço deve possuir **entradas mínimas, critérios de aceite, SLA e evidências de conclusão**. Isso reduz ambiguidades, retrabalho e discussões baseadas apenas em percepção.

A **qualidade** é incorporada diretamente à esteira por meio de:

(1) Integração e entrega contínuas - CI/CD; (2) Análise estática de código; (3) Testes unitários e de contrato; (4) Testes E2E quando necessários; (5) Verificações automáticas de segurança; (6) Testes de compatibilidade; (7) *Quality gates* proporcionais ao risco.

As **decisões** passam a ser apoiadas por indicadores como:

(1) *Lead Time* por tipo de serviço; (2) Percentual de cumprimento de SLA; (3) SLI/SLO; (4) Taxa de falhas após mudanças; (5) Evidências automáticas da entrega.

Estudo de Caso: Modernização de Serviços Digitais em uma Universidade

Como a **Fábrica de Software** organiza a operação, enquanto **Scrum** orienta o trabalho da equipe

1 Cenário

Uma universidade precisa modernizar seus serviços digitais: matrícula on-line, portal do aluno, emissão de documentos e integração com sistemas acadêmicos e financeiros.

- Muitas solicitações chegam ao mesmo tempo
- Prazos e prioridades variam
- É necessário manter qualidade, segurança e previsibilidade



2 Fábrica ≠ Metodologia

Fábrica de Software

- Define o catálogo de serviços
- Padroniza entradas e saídas
- Organiza a esteira de ponta a ponta
- Controla SLA, métricas e quality gates

≠

Metodologia (Scrum / Kanban)

- Organiza o trabalho diário da equipe
- Define cadências, papéis e cerimônias
- Apoia priorização e acompanhamento
- Pode ser usada dentro da Fábrica

A metodologia atua dentro da operação da Fábrica, e não no lugar dela.

3 O que pode ser solicitado?

- **Nova API**
- **Evolução de tela**
- **Correção crítica**
- **Integração externa**
- **Migração de versão**

Cada tipo de serviço possui DoR, DoD, SLA e evidências de conclusão.

4 Fluxo típico da Fábrica

1. Requisitos	2. Análise e Projeto	3. Implementação	4. Testes	5. Implantação	6. Suporte
Demanda registrada e priorizada	Escopo, impacto e critérios de aceite	Pequenos PRs, revisão e automação	Unitários, contrato, segurança e validação	CI/CD, feature flags e rollback	Monitoramento, SLA e lições aprendidas

Quality gates e automação presentes em toda a esteira

5 Resultados do caso

- **-40%** Lead time
- **+30%** Previsibilidade das entregas
- **-50%** Retrabalho
- **-60%** Incidentes em produção

Aprendizado central

A Fábrica de Software cria padronização, governança e escala. Scrum e Kanban ajudam a equipe a executar melhor o trabalho dentro dessa estrutura.

Alinhando o vocabulário: ALM e rastreabilidade

O ***Application Lifecycle Management*** **(ALM)** integra práticas, processos e ferramentas ao longo de todo o ciclo de vida do *software*, conectando **planejamento, desenvolvimento, testes, entrega e operação**.

Seu principal elemento é a **rastreabilidade**, que permite acompanhar uma demanda desde sua origem até seus efeitos em produção.

Com essa visão ponta a ponta, a equipe consegue responder com clareza:

- Por que a mudança foi realizada?
- O que exatamente foi alterado?
- Quem participou da mudança?
- Quais testes e verificações foram executados?
- Qual foi o impacto observado em produção?

Modelos de fábrica:

Fábrica de Serviços

(1) Adequada para grande volume de solicitações semelhantes; (2) Organiza o trabalho por **tipos de serviço padronizados**; (3) Define requisitos mínimos de entrada e critérios verificáveis de saída; (4) Utiliza **SLA, *lead time* e taxa de reabertura**; (5) Favorece previsibilidade, auditoria e comparação entre áreas.

Exemplos: correções, evoluções de tela, novas APIs e integrações recorrentes.

Fábrica orientada a Produto

(1) Indicada quando há maior incerteza e evolução contínua; (2) Utiliza **times estáveis e multidisciplinares**; (3) Trabalha com *backlog* de produto, *discovery* e *delivery*; (4) Valoriza componentização, versionamento e contratos de API/eventos; (5) Usa SLI/SLO, CFR, MTTR e métricas de adoção ou conversão.

Modelos de fábrica:

Modelo orientado a Projeto

(1) Adequado quando existe **prazo fixo, escopo definido ou obrigação regulatória**; (2) Trabalha com orçamento, cronograma e *baseline*; (3) Deve evitar grandes entregas únicas; (4) Utiliza marcos incrementais e critérios de aceite observáveis; (5) Pode adotar *feature toggles*, *rollback* e liberações por ondas.

Indicadores comuns:

(1) Cumprimento de marcos; (2) Desvio de escopo; (3) Defeitos por etapa; (4) SLO associados ao impacto no usuário.

Modelo de Programa/Portfólio

(1) Utilizado quando várias iniciativas possuem dependências entre si; (2) O foco passa a ser **coordenação e priorização**; (3) Torna visíveis dependências entre projetos; (4) Sincroniza janelas de liberação; (5) Distribui capacidade segundo valor, risco e prioridade; (6) Pode utilizar *release trains* e quadros de dependências.

Estudo de Caso: Universidade Digital e os 4 Modelos de Fábrica

Aplicando fábrica de serviços, fábrica orientada a produto, modelo orientado a projeto e programa/portfólio

Cenário

Uma universidade mantém vários sistemas digitais: portal do aluno, AVA, biblioteca, app de saúde universitária, financeiro e integração entre campi. A área de TI recebe demandas rotineiras, evoluções contínuas, projetos com prazo fixo e iniciativas interdependentes.



Triagem das demandas

Entrada única de demandas
Portal, Service Desk ou Kanban

- **Pedido repetitivo** (solicitações semelhantes) → 1. Fábrica de Serviços
- **Evolução contínua** (incerteza, melhorias) → 2. Fábrica orientada a Produto
- **Prazo fixo** (escopo definido) → 3. Modelo orientado a Projeto
- **Dependências entre** iniciativas → 4. Programa / Portfólio

1. Fábrica de Serviços

Quando usar:
alto volume de solicitações semelhantes e recorrentes.

Exemplo no caso:
correções no portal do aluno, novas APIs, ajustes de tela, integrações repetitivas.

Características:
catálogo de serviços, entrada mínima, DoD claro, fluxo padronizado.

Indicadores:
SLA, *lead* time, taxa de reabertura.

2. Fábrica orientada a Produto

Quando usar:
produto digital em evolução contínua e com incerteza.

Exemplo no caso:
evolução do AVA e do app de saúde universitária.

Características:
time estável e multidisciplinar, *backlog*, *discovery* + *delivery*, componentização e contratos de API/eventos.

Indicadores:
SLI/SLO, adoção, conversão, CFR, MTTR.

3. Modelo orientado a Projeto

Quando usar:
prazo fixo, escopo definido ou exigência regulatória.

Exemplo no caso:
implantação de LGPD, migração do sistema financeiro ou *rollout* de nova matrícula.

Características:
cronograma, orçamento, *baseline*, marcos incrementais, *rollout* por ondas, *rollback/feature toggles*.

Indicadores:
cumprimento de marcos, desvio de escopo, defeitos por etapa, SLO do serviço afetado.

4. Programa / Portfólio

Quando usar:
múltiplas iniciativas com dependências entre si.

Exemplo no caso:
integração entre matrícula, biblioteca, AVA, financeiro e autenticação institucional.

Características:
coordenação e priorização, visibilidade de dependências, janelas de liberação, alocação de capacidade por valor e risco.

Indicadores:
dependências resolvidas, previsibilidade de releases, fluxo entre projetos, priorização por valor.

Comparação rápida

	Fábrica de Serviços	Fábrica orientada a Produto	Modelo orientado a Projeto	Programa / Portfólio
Foco principal	Padronização e eficiência	Evolução contínua e valor	Entrega com data e conformidade	Coordenação e priorização
Tipo de demanda	Repetitivas e recorrentes	Incerteza e melhorias	Escopo definido / regulatória	Dependências entre iniciativas
Unidade de organização	Serviços	Produto digital (time estável)	Projeto	Conjunto de iniciativas (portfolio)
Exemplo no caso	Correções e integrações	AVA e app de saúde	Migração LGPD / financeiro	Integração entre sistemas

Ideia-chave

A mesma organização pode usar os 4 modelos.
O segredo está em ter uma entrada única, critérios de triagem e uma base comum de qualidade, automação e métricas.

Combinar modelos com governança leve:

Combinar modelos com **governança leve** significa utilizar uma **porta de entrada única para todas as demandas** e encaminhar cada solicitação ao fluxo mais adequado, sem criar excesso de aprovações ou estruturas paralelas.

A triagem pode partir de **três** perguntas:

A demanda é **rotineira e repetível**?

→ segue para o **Catálogo de Serviços**, com entrada mínima e DoD claros.

A demanda envolve **descoberta, hipóteses ou evolução contínua**?

→ segue para o **fluxo de Produto**, com *roadmap*, épicas e *backlog*.

Existe **data inegociável, obrigação legal ou marco regulatório**?

→ segue como **Projeto**, com marcos incrementais e cronograma definido.

Estudo de Caso: Governança Leve para Organizar Demandas em uma Universidade

Como uma **porta única de entrada** direciona cada solicitação para Catálogo de Serviços, Produto ou Projeto.

1 Cenário



Uma universidade recebe solicitações de várias áreas: secretaria acadêmica, financeiro, biblioteca, TI e coordenações. As demandas chegam misturadas: ajustes simples, melhorias contínuas e iniciativas com prazo regulatório.

- Muitas solicitações simultâneas
- Naturezas diferentes de demanda
- Necessidade de rapidez, clareza e controle

2 Governança leve

Em vez de criar múltiplos comitês e fluxos paralelos, a universidade adota uma **porta única de entrada** para todas as demandas. A triagem decide o fluxo adequado com regras simples e objetivas.

Porta única de entrada
Todas as demandas em um só canal

Triagem rápida
Decisão com base em três perguntas

Menos burocracia
Sem comitês paralelos

Mais clareza
Cada demanda no fluxo mais adequado

3 As 3 perguntas da triagem

Nova solicitação
Chega pela porta única de entrada

1 É uma demanda rotineira e repetível?
→ **Catálogo de Serviços**
Serviços padronizados e bem definidos

Exemplos de demandas: Nova API; Evolução de tela; Correção crítica; Integração pxterna; Migração de versão

DoR, DoD, SLA e evidências claras.

2 Envolve descoberta, hipóteses ou evolução contínua?
→ **Fluxo de Produto**
Evolução contínua com foco em valor

Exemplos de demandas: Portal do aluno; App institucional; Experiência digital

Roadmap, épicos, backlog e entregas incrementais.

3 Existe data inegociável, obrigação legal ou marco regulatório?
→ **Projeto**
Iniciativas com prazo e escopo definidos

Exemplos de demandas: Adequação à LGPD; Integração com sistema do MEC; Implantação com prazo fixo

Marcos, cronograma e encampanhamento formal.

4 Exemplo prático

Catálogo
A secretaria pede emissão automática de declaração de matrícula.

Resultado: serviço padronizado e rápida execução.

Produto
A pró-reitoria quer melhorar continuamente o portal do aluno.

Resultado: priorização por valor e evolução contínua.

Projeto
Nova exigência regulatória exige adequação do sistema financeiro até data definida.

Resultado: cronograma, responsáveis e entregas por fase.

5 Benefícios obtidos

- **-35%** tempo de triagem
- **+40%** clareza sobre o fluxo correto
- **-30%** retrabalho
- **+25%** previsibilidade das entregas

Aprendizado central
A governança leve não elimina métodos de trabalho. Ela organiza a entrada e o encaminhamento das demandas, permitindo combinar Catálogo, Produto e Projeto com menos burocracia e maior efetividade.

Bons estudos!

Siga em frente com a



www.unicesumar.edu.br